

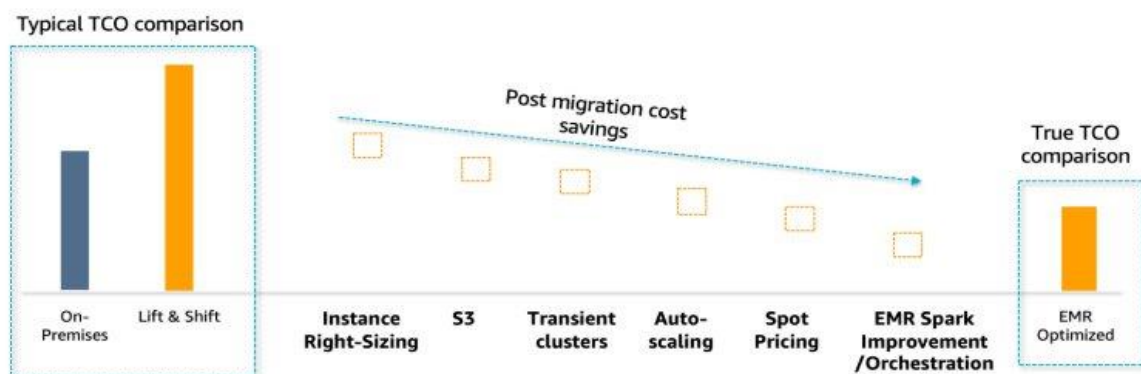
Introducing the AWS ProServe Hadoop Migration Delivery Kit TCO tool

When [migrating Hadoop workloads to Amazon EMR](#), it's often difficult to identify the optimal cluster configuration without analyzing existing workloads by hand. To solve this, we're introducing the Hadoop migration assessment Total Cost of Ownership (TCO) tool. You now have a Hadoop migration assessment TCO tool within the AWS ProServe Hadoop Migration Delivery Kit (HMDK). The self-serve HMDK TCO tool accelerates the design of new cost-effective [Amazon EMR](#) clusters by analyzing the existing Hadoop workload and calculating the total cost of the ownership (TCO) running on the future Amazon EMR system. The Amazon EMR TCO report with the new Amazon EMR design can demonstrate the Amazon EMR migration with detailed cost saving and business benefits.

In this post, we introduce a use case and the functions and components of the tool. We also share case studies to show you the benefits of using the tool. Finally, we show you the technical information to use the tool.

Use case overview

Migrating Hadoop workloads to Amazon EMR accelerates big data analytics modernization, increases productivity, and reduces operational cost. Refactoring coupled compute and storage to a decoupling architecture is a modern data solution. It enables compute such as EMR instances and storage such as [Amazon Simple Storage Service](#) (Amazon S3) data lakes to scale. For various Hadoop jobs, customers have bespoke deployment options of fully managed Amazon EMR, [Amazon EMR on Amazon EKS](#), and [EMR Serverless](#). The optimized future EMR cluster yields the same results and values with much lower TCO compared to the source Hadoop cluster. But we need a TCO report to showcase the cost saving details, as shown in the following figure.



Typically, the commencement of a Hadoop migration needs Hadoop experts to spend weeks or even months to assess current Hadoop cluster workloads towards a plan for subsequent migration. This could delay the project from being accepted without a good TCO report.

To accelerate Hadoop migrations and mitigate the workload assessment efforts by SMEs, AWS ProServe created the Hadoop migration assessment TCO tool within the AWS ProServe Hadoop Migration Delivery Kit.

Introduction to the HMDK TCO tool

As a Hadoop migration accelerator, the HMDK TCO tool has three components:

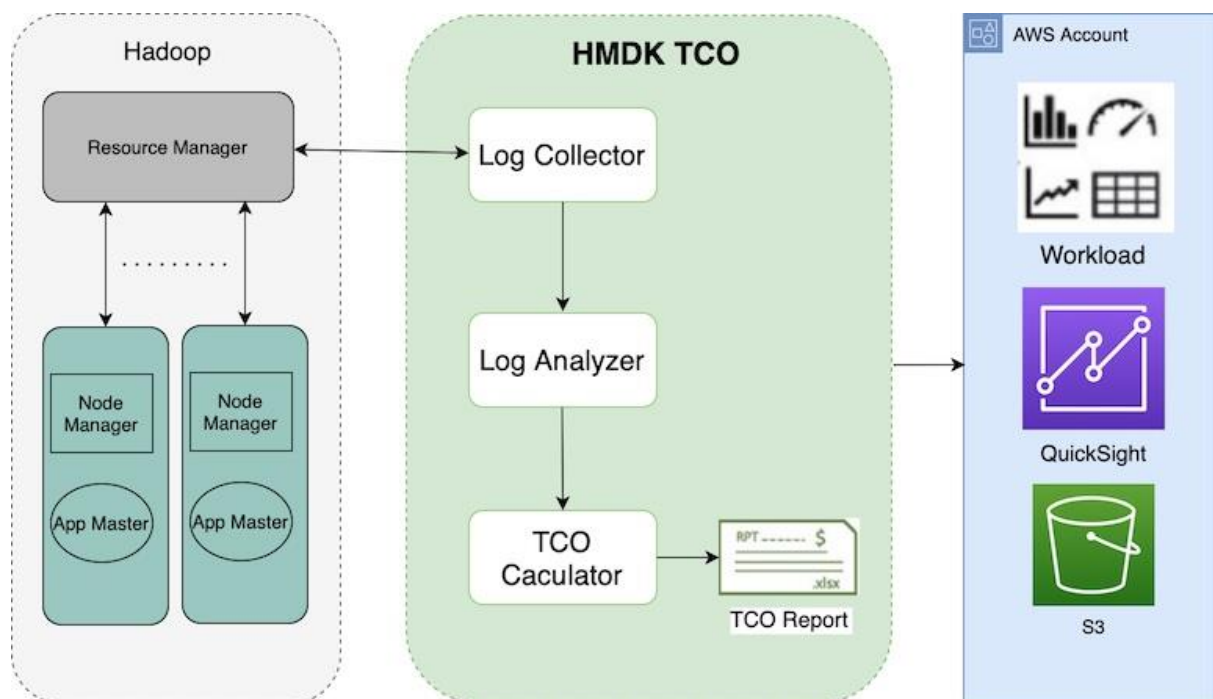
- YARN log collector – Retrieves the existing workload logs from YARN Resource Manager
- YARN log analyzer – Provides a deep time-based insight on different aspects of the jobs
- TCO calculator – Generates a 3-year or 1-year TCO calculated automatically

The self-serve HMDK TCO tool is available for download on [GitHub](#).

Using the tool consists of three steps:

1. First, the YARN Log collector communicates with the current Hadoop system to retrieve YARN logs.
2. With the collected YARN logs, the next step is to use the YARN log analyzer and set up the log analyzer stack using [AWS CloudFormation](#). The results of the log analyzer reveal Hadoop workload insights with various views and metrics of the Hadoop applications shown in [Amazon QuickSight](#) dashboards, which leads to the design of a future EMR cluster.
3. Lastly, the TCO calculator generates the TCO report by simulating hourly resource usage of a future EMR cluster. To accelerate Hadoop migration assessment, the TCO report provides crucial information and values for your business stakeholders to make a buy-in decision.

The following diagram illustrates this architecture.



The Hadoop workload insights enable you to design a well-architected EMR cluster to achieve performance and cost-effectiveness in an agile way. For conducting well-architected designs, you need to deliberate between various system specifications of an EMR cluster and multiple cost considerations.

The system specifications are as follows:

- Number of EMR clusters – Amazon EMR enables you to run multiple elastic clusters in the AWS Cloud to serve the same purpose of a shared static Hadoop cluster on premises

- Types of EMR cluster (persistent or transient) – Design your system to keep minimum persistent clusters to save cost
- Instance types and configuration (memory, vCore, and so on) – Choose the right instance for your job
- Resource allocation for applications and cluster utilization – Based on the on-premises workload analysis, design effective resource allocation and efficient resource utilization in future EMR clusters

The cost considerations are as follows:

- Latest price list (from thousands of available EC2 instances available) – The HMDK TCO tool makes the price calculation with [Amazon Elastic Compute Cloud](#) (Amazon EC2) instance types, configurations, and their prices.
- Amazon S3 storage cost (standard, Glacier, and so on) – Data replication is no longer required for reliability. You can use tiered storage in Amazon S3 for cost savings.

YARN log collector

The HMDK TCO tool enables a simple way to capture Hadoop YARN logs, which include the Hadoop job runs statistics and the corresponding resource usages. The following screenshot is an example of a YARN log.

id	name	queue	user	applicationType	startedTime	elapsedTime	memorySeconds	vcoreSeconds
application_1624493280935_0001	hive	HIVE-74a34200-3b44-412a-bb40-be8b4dbd4dd4	default	TEZ	1624493323938	315695	1846431	315
application_1624493280935_0003	hive	HIVE-0dc8c910-2046-472c-8aa0-7a3be28f28cf	default	TEZ	1624493341927	311478	1818487	310
application_1624493280935_0002	hive	HIVE-8de01ff8-0e04-4441-8842-22c58fe2f8c2	default	TEZ	1624493326985	313140	1832957	313
application_1619050035242_0001	hive	HIVE-83f3f193-36ea-46e9-bdcb-31f1280f6597	default	TEZ	1619050075409	538517	3150194	537
application_1619050035242_0002	hive	HIVE-97d60a62-be51-4ffd-a2bb-78836355d252	default	TEZ	1619050080590	533069	3119379	532
application_1619050035242_0003	hive	HIVE-0759de54-be9d-4d8e-9f54-681509df1909	default	TEZ	1619050090775	520859	3047971	520
application_1618704380064_0002	hive	HIVE-e133d884-f6b8-4402-b125-e1c509f1a7ac	default	TEZ	1618704458189	361269	2115462	361
application_1618704380064_0001	hive	HIVE-99c2b7a4-6c9e-4b68-9ad1-2c13fefe5fbc	default	TEZ	1618704452236	358064	2094129	357
application_1618704380064_0003	hive	HIVE-945a5faa-b6df-463f-9938-4f4646ac44e0	default	TEZ	1618704458147	341157	1997399	341
application_1628208388867_0003	hive	HIVE-e416d64e-8a4f-4384-852f-65afe33cea2b	default	TEZ	1628208443426	314062	1813409	309
application_1618669661076_0001	hive	HIVE-529fc7dc-e798-4f70-9de9-865967b47d57	default	TEZ	1618669697562	315208	1833062	313

The tool supports HTTPS protocol to communicate with YARN Resource Manager. The tool transports the JSON YARN logs as the inputs to a Python parser, which converts the YARN logs from JSON to CSV format. The new CSV formatted logs are the standard input files for the YARN log analyzer.

For more information, see the [GitHub repo](#).

YARN log analyzer and optimized design use cases

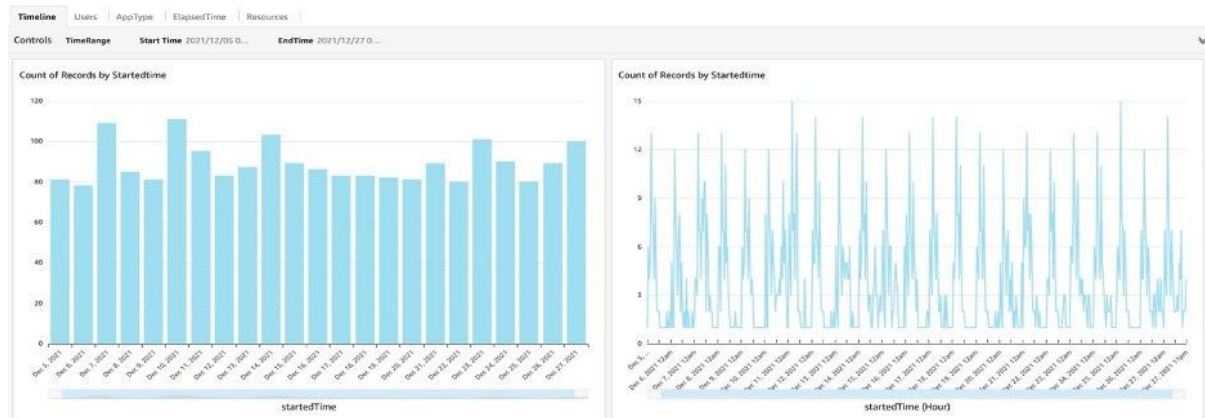
With the log, we can follow up the steps in the [TCO yarn-log-analysis README](#) file to use AWS CloudFormation to set up QuickSight resources.

The HMDK TCO log analyzer generates a QuickSight dashboard on various metrics:

- Job timeline – How many jobs are running at one time
- Job user – Breakdown of users and queues
- Application type and engine type – Breakdown by application types (Spark, Hive, Presto) and run engine type (MapReduce, Spark, Tez)

- Elapsed time – The time span of completing an application
- Resources – Memory and CPU

The following screenshot shows an example dashboard.



The QuickSight dashboards exhibit insights based on consecutive YARN logs collected in a long-enough period of time (for example, a 2-week window). The insights from the logs reveal the application types, users, queues, running cadence, time spans, and resource usages. The data also helps you discover daily batch jobs or ad hoc jobs, long-running jobs, and resource consumption. These insights help you design the right clusters, such as transient clusters or baseline permanent clusters, and choose the right EC2 instance for memory- or compute-intensive jobs. With the log analyzer results, the TCO tool automatically calculates the TCO of a future EMR cluster.

Let's see some real customer use cases in the following sections.

Case 1: Use transient and persistent clusters wisely

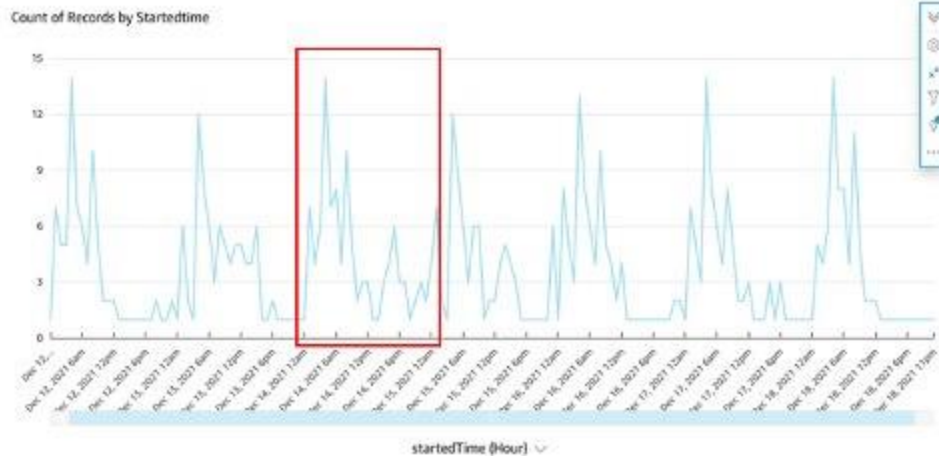
For this use case, a customer in the financial sector has an 11-node Hadoop cluster.

The QuickSight timeline dashboard shows the peak time job runs because of the daily batch job. This guides us to design two clusters for fulfilling the existing workloads. When we keep a persistent cluster at a minimal size, we can have the transient EMR cluster to handle the batch style job around the peak time.

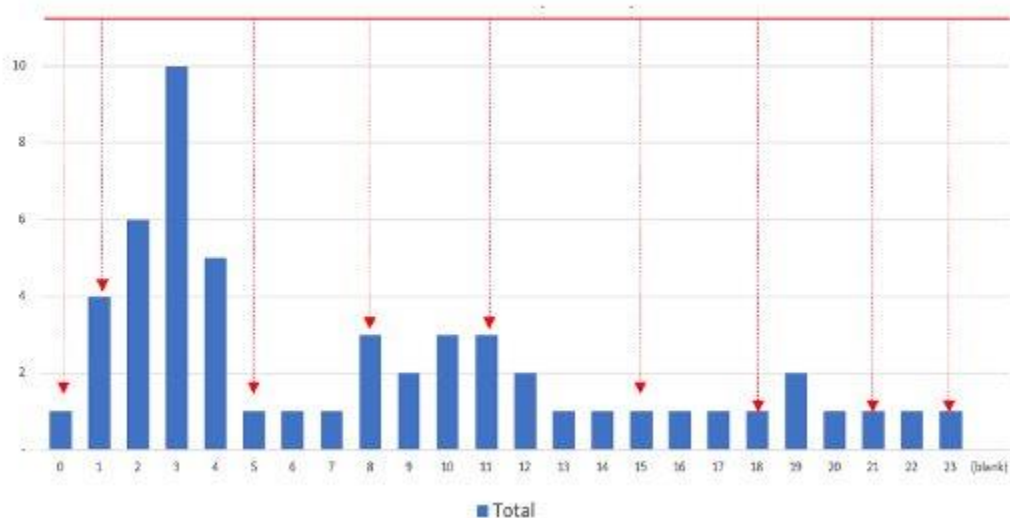
Therefore, we designed the clusters to have a persistent cluster with 2 data nodes, while transient nodes can scale from 0–10 between the hours of 1:00 AM and 4:00 AM.

The following figure illustrates this design.

AS-IS Workload Pattern by Hourly



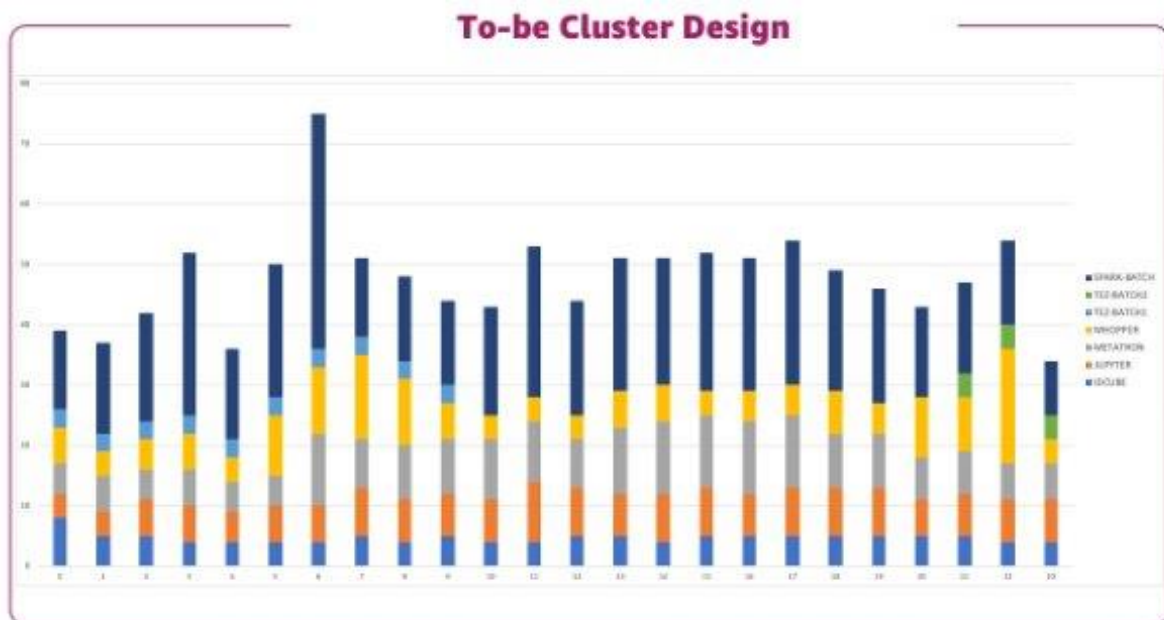
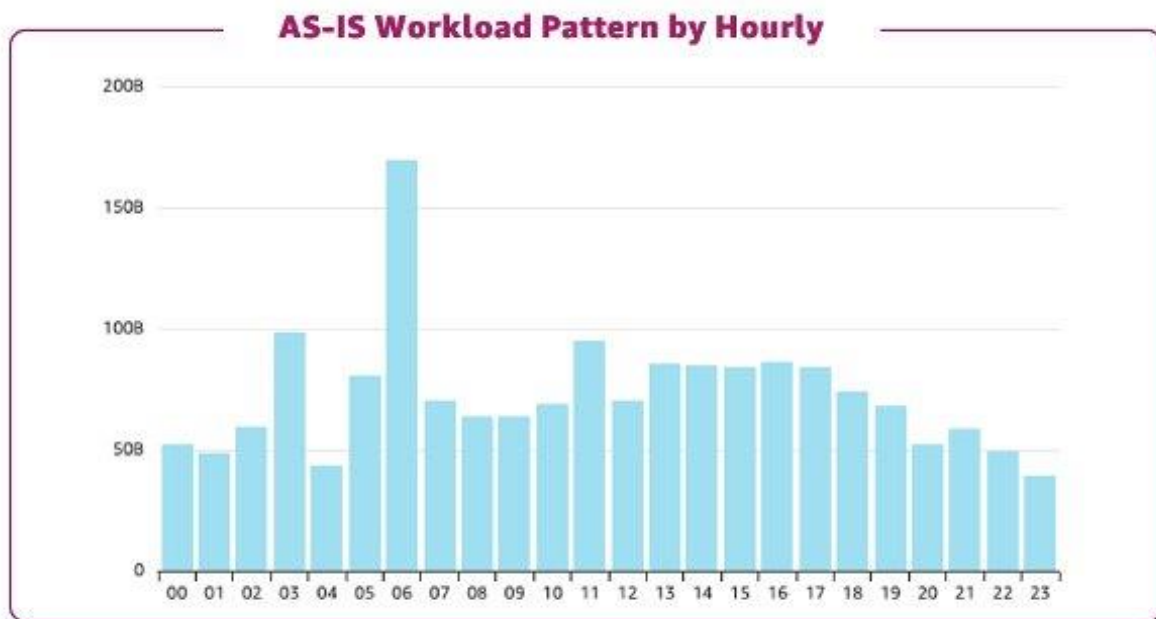
To-be Cluster Design



This balanced design using transient and persistent clusters resulted in a cost savings of about 80% compared to a lift-and-shift design.

Case 2: Identify Hadoop queue usage and long-running jobs to design multiple clusters and optimized runs

For our next use case, a company runs 196 nodes using Hadoop 3.1 with jobs like Hive, Spark, and Kafka. The Hadoop default queue and four other queues were used to group various workloads. As illustrated in the following figure, some very long-running jobs are seen in the shared cluster, resulting in queued jobs that have resource competition and unbalanced resource allocation.



The QuickSight user dashboard guides us through the queue usage, the elapsed time dashboard guides us through the long-running jobs, and the resource dashboard guides us through the memory and vCore usage for the jobs.

Therefore, we design a solution to transfer queue jobs to run in separated clusters, and the default queue jobs are split to run in different clusters. By identifying the long-running jobs and understanding the resource needs, we could design a cluster to run such jobs more efficiently.

This design allows the job to run faster and the clusters to be used more efficiently with a cost savings benefit.

Cluster design

The HMDK TCO tool provides a cluster design template like the following example.

clusterOrder	clusterType	clusterName	queue	user	appType	startHour	endHour
1	T	SPARK_ETL	etl	hadoop	spark	11	14
2	P	Persistent1	ad-hoc, etl, default	hadoop,user1,user2, user3,user4	spark,tez	0	23

Here we have two clusters, one transient and one persistent, to handle the Spark and Tez jobs accordingly. The starting and ending hour for each cluster can be determined from the log analysis. With this cluster design, we can get the hourly workload resource usage forecast. Then the TCO calculator gets all the information needed to generate costs based on the TCO simulation variables you choose.

TCO calculator

The HMDK TCO calculator is a component guiding the EMR cluster design by using the EMR design template. Then it generates the hourly aggregated resource usage forecast using a Python program. The component provides guidelines and an Excel template to input system and cost specification parameters. The component has the logic with a built-in Amazon EMR price list. The 1-year and 3-year TCO cost can be automatically generated by the macro-enabled Excel TCO template.

The following figure shows the details of our HMDK TCO simulation.

Region	Asia Pacific (Seoul)					
EC2 Type	Spec		vCores	Memory(G)	EBS Type	Disk Size(G)
	Master	m4.large	2	15	HDD(ST1)	512
	Task/Core	m5.xlarge	4	32	HDD(ST1)	512
EMR High Availability	P	Yes	P : Persistent T: Transient			
	T	No				
Hive to Spark Conv. Effect		0%				
EC2 Discount(EDP)		10%	EBS Discount(EDP)		10%	
S3 Volume Discount		10%	Local Currency Rate		1180	

AutoScaling	ON	Mem Util. Factor	0.85	* Do not Change If you don't know the meaning!! (Default:0.85)		
-------------	----	------------------	------	--	--	--

EMR EC2 Task/Core Pricing Ratio and Price / Hour						
Persistent	Type	on-demand	RI(3-year)	RI(1-year)	Spot	Check
	Ratio	40%	40%	40%	40%	160%
transient	Type	on-demand	RI(3-year)	RI(1-year)	Spot	Check
	Ratio	60%	0%	0%	40%	100%

S3 Storage Ratio					
Type	Standard	Standard-IA	S3 Glacier	S3 Glacier Deep Archive	Check
Ratio(%)	40%	40%	10%	10%	100%
Size(TB)*	20	20	5	5	50

* Do not input size - automatically calculated

* Do not input size - automatically calculated

The following figures show the TCO report.

1) Optimized EMR EC2

Clusters	-T	1 year TCO		
		TCO(\$)	TCO-Discount(\$)	TCO-Discount-Local
P		\$ 59,870	\$ 53,883	63,581,429
DW1		\$ 55,068	\$ 49,561	58,481,831
DW2		\$ 4,802	\$ 4,322	5,099,598
T		\$ 546	\$ 491	579,429
BATCH1		\$ 546	\$ 491	579,429
Grand Total		\$ 60,415	\$ 54,374	64,160,859

2) Storage

	Storage Type	1 year TCO		
		TCO(\$)	TCO-Discount	TCO-Discount-Local
S3	standard	\$ 6,144	\$ 5,530	6,524,928
	Infrequent Access	\$ 3,391	\$ 3,052	3,601,760
	S3 Glacier	\$ 6,144	\$ 5,530	6,524,928
	S3 Glacier Deep	\$ 123	\$ 111	130,499
EB5	HDD(ST1)	\$ 124	\$ 112	131,670
Grand Total		\$ 15,926	\$ 14,334	16,913,785

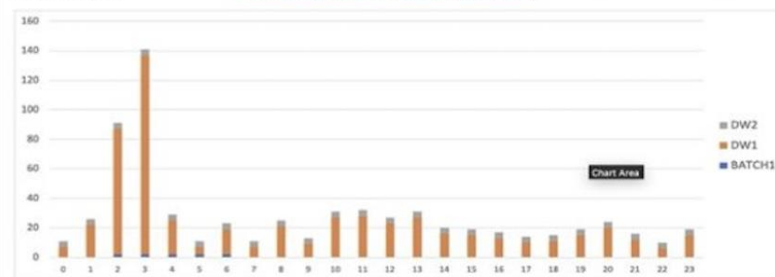
* Lift-and-Shift (EMR EC2, by Memory)

Cluster	Instance Type/ # Node	1 year TCO		
		3-Year RI TCO(\$)	TCO-Discount(\$)	TCO-Discount-Local
Single	m5.xlarge	\$ 133,187	\$ 119,868	141,444,636
Persistent Cluster	14			

* EC2 type Auto recommendation Logic = MIN(EC2 Memory >= on-premise Memory)

3) EMR Nodes/hourly

* Auto Refresh When change TCO Simulation Variables(1~3sec)



TCO tool engagement outcomes

In this section, we share some of the engagement outcomes from customers after using the TCO tool for 1–2 weeks. Additionally, with the TCO tool, we can refactor on-premises Hadoop clusters to EMR clusters utilizing Amazon S3 as a data lake. The modern data solution of migrating to Amazon EMR provides unlimited scalability with operational efficiency and cost savings.

The following table illustrates four case studies of some engagements using the tool.

Case#	Case Description	Engagement Outcome
-------	------------------	--------------------

1

Pressured by the Hadoop License, they migrated to AWS using Amazon EMR and used Spark for replacing Hive. They designed the new EMR clusters using a balanced design of transient and persistent clusters.

They can get job insights through the tool and design the new EMR clusters to fulfill the existing workloads, and expect to achieve 80% cost savings and six times performance enhancement.

2

Their goal was to migrate a Hadoop cluster with over 1,000 nodes from HDFS to Amazon S3 and Hive to Spark, and redesign the cluster using a balanced design of transient and persistent clusters.

They can get job insights and redesign the cluster with a 1-year TCO of the optimized redesign architecture expected to have 64% cost savings.

- 3 Their goal was to migrate to Hadoop 3.1. They transferred the Hadoop queue-based job, which shared the same cluster, to two transient clusters and five persistent clusters with optimized resource usage for each job run, and handled long-running jobs faster. They can get Amazon EMR TCO results quickly in 2 weeks. Customers get insights on their workloads and long-running jobs and get the job done faster and cheaper.
- 4 Their goal was to migrate from Hive 1 to Spark and design an auto scaling EMR cluster. They can get Amazon EMR TCO results in 1 week. They're expecting to see 75% cost savings on the redesigned EMR clusters and 10 times on performance improvement.